Object Storage

1. Overview

Object storage stores data as independent **objects** instead of files in folders or fixed disk blocks.

Each object usually contains:

- **Data** (file/content)
- **Metadata** (size, type, owner, tags)
- **Unique ID / Key**

Objects are commonly accessed using **HTTP APIs**.

Example:

```
bucket: app-assets



Key:

images/user101.png

videos/intro.mp4

docs/report.pdf
```

2. Core Idea

Instead of mounting a disk, applications call APIs:

```None
PUT object

GET object

DELETE object

LIST objects
```

Usually over HTTPS.

3. Popular Examples

- Amazon Web Services Amazon S3
- Google Cloud Google Cloud Storage
- Microsoft Azure Blob Storage
- MinIO

4. Flatter Hierarchy Than File Storage

Looks like folders, but usually keys are just names.

```None
photos/2026/april/a.jpg
```

This may appear hierarchical, but it is often just a key string.

No true filesystem tree required.

5. Why Used So Much

Massive scalability
Durable storage
Cheap for large data
Accessible over internet/API
Great for static assets
Easy backup/archive

6. Best Use Cases

Use object storage for:

- Images
- Videos
- PDFs
- User uploads
- Backups
- Logs archives
- Static websites
- Data lake files
- ML datasets

7. Why Good for Static Data

Objects are often treated as immutable.

Instead of editing in place:

```None
old file → replace with new object
```

Good for:

- Media
- Versioned assets
- Reports
- Snapshots

8. Why Writes Can Be Slower

Compared with RAM cache or local SSD:

- Network hop required
- API call overhead
- Replication durability steps
- Higher latency than local disk

So object storage is not ideal for hot transactional workloads.

9. Object Storage vs File Storage

Feature	File Storage	Object Storage
Interface	Filesystem	HTTP/API
Hierarchy	Real folders	Flat namespace

Edit file in place	Yes	Usually replace object
Scale	Moderate	Massive
Web delivery	Indirect	Excellent

10. Object Storage vs Block Storage

Feature	Block Storage	Object Storage
Latency	Low	Higher
Random writes	Excellent	Weak
Mount as disk	Yes	No
Massive archive	Moderate	Excellent
Best for	DB disks	Static content

11. Example Architecture

```None
User uploads image

→ App Server

→ Store image in S3

→ Save URL/path in database
```

Database stores metadata only.

12. CDN Integration

Very common:

```None
Object Storage + CDN
```

Example:

Amazon Web Services Amazon CloudFront + Amazon S3

Used for fast global delivery.

13. Metadata Example

Each object may include:

```None
content-type=image/png

size=2MB

owner=user101

created_at=...

tags=avatar
```

14. Durability Features

Object stores often provide:

- Multi-copy replication
- Versioning
- Lifecycle rules
- Cross-region replication
- Backup retention

15. Security Features

Common controls:

- IAM / access policies
- Signed URLs
- Encryption at rest
- TLS in transit
- Bucket permissions

16. Trade-offs

Higher latency than local disk
Poor for DB random writes
Rename/update semantics differ
Many small objects can be inefficient

17. Interview Answer Example

Q: Where should user-uploaded videos be stored?

I’d use object storage like S3 because it scales well, is durable, cost-effective, and integrates easily with CDN delivery.

18. Short Revision Notes

```None
Object Storage:

- Stores objects via HTTP APIs

- Flat namespace

- Great for static data



Best for:

images, videos, backups



Examples:

S3, GCS, Azure Blob
```

19. One-Line Summary

Object storage stores immutable-like data objects accessible through APIs, making it ideal for scalable, durable, and cost-effective storage of static files, media, backups, and large unstructured datasets.